



ESCUELA DE
INGENIERÍA EN CIENCIAS Y SISTEMAS
FACULTAD DE INGENIERÍA
UNIVERSIDAD DE SAN CARLOS DE GUATEMALA



Día, Fecha:

DIA, DD/MM/2026

Hora de inicio:

00:00

Sistemas Operativos 2 [A]

NOMBRE_ESTUDIANTE

SISTEMAS OPERATIVOS 2

Escuela de Ingeniería de Ciencias Y Sistemas
Facultad de Ingeniería
Universidad de San Carlos de Guatemala

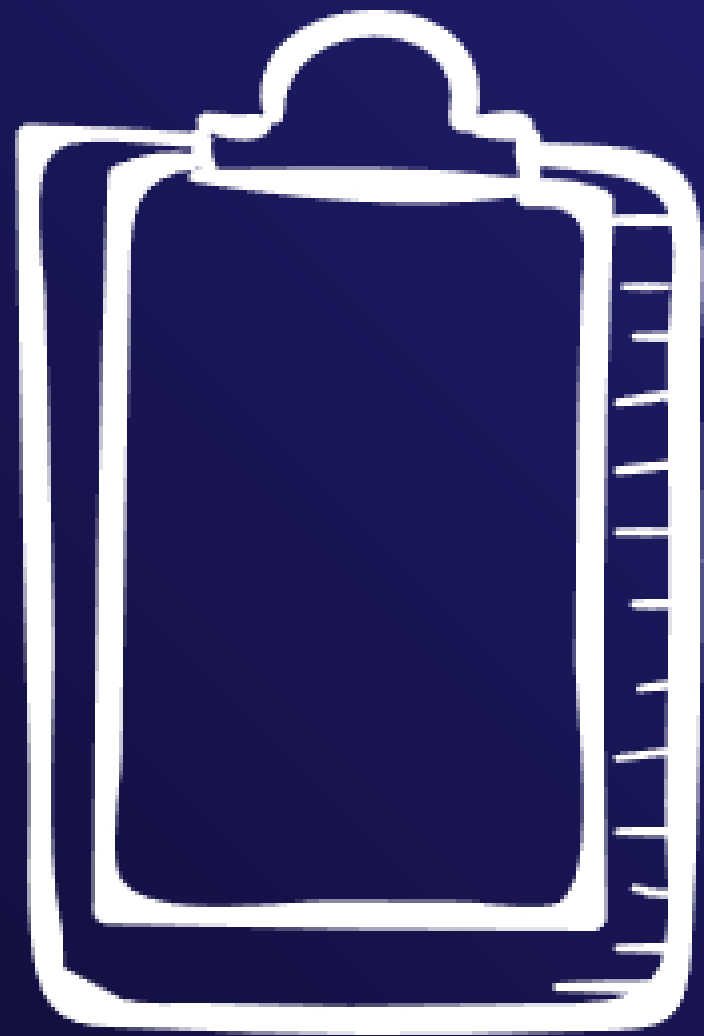


Anuncios Importantes



- **Anuncio 1**
- **Anuncio 2**
- **Anuncio 3**

Anuncios Importantes



1. **Actividad**
2. **Actividad**
3. **Actividad**

Competencia(s) que desarrollaremos



- Entender el marco de referencia o estructura lógica general de un sistema operativo, que le permita la utilización, análisis y diseño de sistemas operativos.
- Usar óptimamente los recursos y API de los sistemas operativos, en el desarrollo de aplicaciones



Valor(es) de la Semana

Equidad

La planificación/programación de CPU se trata de darle el tiempo adecuado a cada proceso en ejecución, muchas veces no se trata de igualdad y la igualdad no es lo óptimo, sino de equidad, que en términos generales significa darle a cada uno el tiempo que necesita para avanzar tanto como el resto.



Unidad 02

Gestión de Memoria



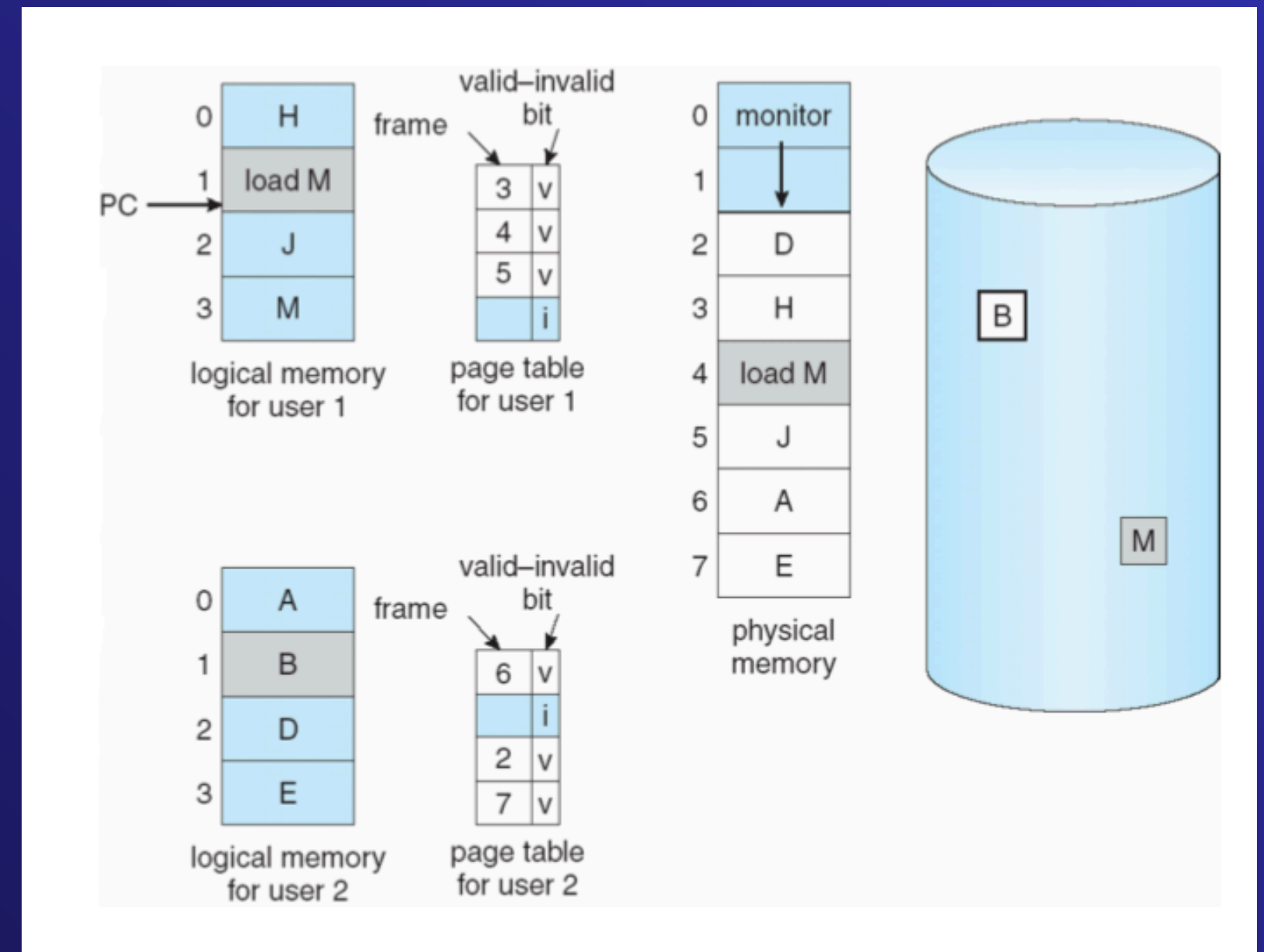


Memoria Virtual: Algoritmos de Reemplazo de Páginas

Algoritmos de Reemplazo de Páginas

¿Por qué se necesitan algoritmos de reemplazo?

- La memoria física (RAM) es limitada y mucho más pequeña que el espacio virtual total de todos los procesos activos.
- Cuando un proceso necesita una página que no está en RAM y no hay marcos libres, el SO debe desalojar una página existente para hacer espacio.
- La elección de qué página desalojar determina cuántos futuros fallos de página se producirán.
- Un mal algoritmo puede degradar severamente el rendimiento del sistema.
- Este problema es equivalente al de cachés en hardware: maximizar los "aciertos" y minimizar los "fallos".



Conceptos Previos Necesarios

- Marco (frame): división física de la RAM de tamaño fijo donde se carga una página.
- Página: unidad lógica del espacio virtual de un proceso, mismo tamaño que un marco.
- Cadena de referencias: secuencia de accesos a páginas que hace un proceso. Ejemplo: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5.
- Fallo de página (page fault): la página referenciada no está en ningún marco cargado en RAM.
- Número de marcos: cantidad de marcos físicos disponibles para ese proceso. A mayor cantidad, menos fallos (en general).

Page reference: 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 3

No. of Page frame - 4

7	0	1	2	0	3	0	4	2	3	0	3	2	3
			2	2	2	2	2	2	2	2	2	2	2
		1	1	1	1	1	4	4	4	4	4	4	4
	0	0	0	0	0	0	0	0	0	0	0	0	0
7	7	7	7	7	3	3	3	3	3	3	3	3	3
Miss	Miss	Miss	Miss	Hit	Miss	Hit	Miss	Hit	Hit	Hit	Hit	Hit	Hit

Total Page Fault = 6

Here LRU has same number of page fault as optimal but it may differ according to question.

FIFO — First In, First Out

La página que entró primero a memoria es la primera en ser reemplazada.

Se mantiene una cola FIFO con el orden de llegada.

Ventaja: extremadamente simple de implementar.

Desventaja principal — Anomalía de Bélády (1969): al aumentar el número de marcos, el número de fallos de página puede aumentar en lugar de disminuir. Contraintuitivo y problemático.

FIFO (First In, First Out) – Ejemplo

Cadena de referencias: 1 2 3 4 1 2 5 1 2 3 4 5 | Número de marcos: 3

Referencia	1	2	3	4	1	2	5	1	2	4	5
Marco 1	1	1	1	4	4	4	5	5	5	5	5
Marco 2	–	2	2	2	1	1	1	1	1	3	3
Marco 3	–	–	3	3	3	2	2	2	2	4	4
Fallo de página	✓	✓	✓	✓	✓	✓	✓	–	✓	✓	✓

- ✓ = Fallo de página
- = No hay fallo (la página ya estaba en memoria)



Resultado:

9 fallos de página

Explicación rápida:

- Se mantiene una cola FIFO con el orden de llegada.
- Cuando ocurre un fallo, se reemplaza la página que lleva más tiempo en memoria.

LRU — Least Recently Used

- Reemplaza la página que no ha sido accedida por más tiempo.
- Se basa en el principio de localidad temporal: lo que se usó recientemente, probablemente se usará pronto de nuevo.
- No sufre la anomalía de Bélády — más marcos siempre significa igual o menos fallos.
- Implementaciones posibles:
 - Contador: cada página guarda el tiempo del último acceso; se reemplaza la de menor contador. Costo: $O(n)$ en búsqueda.
 - Stack/Pila: cada acceso mueve la página al tope; la del fondo es la LRU. Costo: actualización en cada referencia.
- En la práctica pura es costoso; se usan aproximaciones hardware como el bit de referencia.
- Mismo ejemplo → 8 fallos (mejor que FIFO).

LRU (Least Recently Used) – Ejemplo

Cadena de referencias: 1 2 3 4 1 2 5 1 3 4 5 | Número de marcos: 3

Referencia	1	2	3	4	1	2	5	1	2	4	5
Marcos	1	1	1	4	4	4	4	1	1	1	1
Marcos 2	–	2	2	2	1	1	5	5	2	2	4
Marcos 3	–	–	3	3	3	2	2	2	3	3	5
Fallo de página	✓	✓	✓	✓	✓	–	✓	✓	✓	✓	–

✓ = Fallo de página (la página no estaba en memoria)
– = No hay fallo (la página ya estaba en memoria)

¿Cómo funciona LRU?

Cuando ocurre un fallo, se reemplaza la página que tiene más tiempo sin ser referenciada (la “menos reciente”).



Resultado:

8

fallos de página
(mejor que FIFO: 9 fallos)

Detalle de los reemplazos (fallos):

- Ref 1: fallo (carga 1)
- Ref 2: fallo (carga 2)
- Ref 3: fallo (carga 3)
- Ref 4: fallo → reemplaza 1 (la menos reciente) por 4
- Ref 1: fallo → reemplaza 2 (la menos reciente) por 1
- Ref 5: fallo → reemplaza 3 (la menos reciente) por 5
- Ref 1: fallo → reemplaza 4 (la menos reciente) por 1
- Ref 2: fallo → reemplaza 5 (la menos reciente) por 2
- Ref 3: fallo → reemplaza 1 (la menos reciente) por 3
- Ref 4: fallo → reemplaza 2 (la menos reciente) por 4
- Ref 5: no hay fallo (ya está en memoria)

Aproximaciones a LRU

Dado que LRU exacto requiere hardware costoso, los SO usan aproximaciones:

- Algoritmo del Reloj (Clock / Segunda Oportunidad):
 - Cada página tiene un bit de referencia (R).
 - Un puntero recorre las páginas en círculo.
 - Si $R=1 \rightarrow$ se pone $R=0$ y se avanza (segunda oportunidad).
 - Si $R=0 \rightarrow$ esa página se reemplaza.
 - Simula LRU con bajo costo.
- NRU (Not Recently Used): Usa dos bits: Referenciado (R) y Modificado (M).
 - Clasifica páginas en 4 clases (00, 01, 10, 11).
 - Se reemplaza la de menor clase disponible.
- Aging Algorithm: desplazamiento de bits para simular una ventana de tiempo reciente.

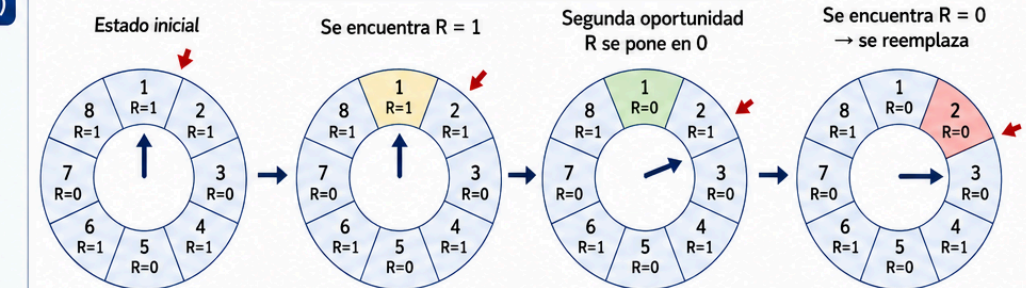
APROXIMACIONES A LRU

LRU exacto requiere hardware costoso. Los sistemas operativos usan aproximaciones.

1 Algoritmo del Reloj (Clock / Segunda Oportunidad)

- Cada página tiene un bit de referencia (R).
- Un puntero recorre las páginas en círculo.
- Si $R=1 \rightarrow$ se pone $R=0$ y se avanza (segunda oportunidad).
- Si $R=0 \rightarrow$ esa página se reemplaza.
- Simula LRU con bajo costo.

Ejemplo de funcionamiento del Algoritmo del Reloj



2 NRU (Not Recently Used)

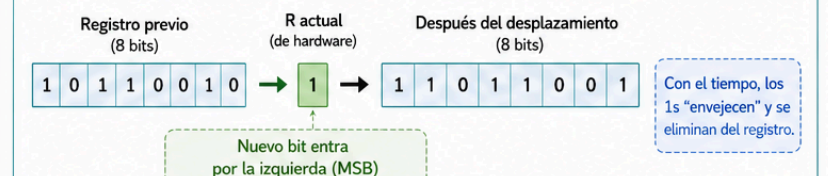
- Usa dos bits: Referenciado (R) y Modificado (M).
- Clasifica las páginas en 4 clases según (R, M).
- Se reemplaza la página de la clase más baja disponible.

Clase	R	M	Descripción	Prioridad
0	0	0	No referenciada, no modificada	Más baja (mejor candidata)
1	0	1	No referenciada, modificada	2
2	1	0	Referenciada, no modificada	3
3	1	1	Referenciada, modificada	Más alta (peor candidata)

Idea clave:
Se elige la página con la clase de menor prioridad numérica disponible.

3 Aging Algorithm (Envejecimiento)

- Simula una ventana de tiempo reciente usando un registro por página.
- Periódicamente:
 - 1 Se desplaza el registro a la derecha.
 - 2 El bit de referencia R actual entra por el bit más significativo (MSB).
- Las páginas con 1s más recientes tienen valores mayores.
- Se reemplaza la página con el valor más bajo.



Algoritmo Óptimo (OPT / MIN)

Algoritmo Óptimo (OPT / MIN)												
Cadena de referencias:												Número de marcos: 3
Instante (t)	1	2	3	4	5	6	7	8	9	10	11	12
Referencia	1	2	3	4	1	2	5	1	2	3	4	5
Marco 1	1	1	1	4	4	4	5	5	5	3	3	3
Marco 2	-	2	2	2	1	1	1	1	2	2	4	4
Marco 3	-	-	3	3	3	2	2	-	-	-	-	5
Fallo de página	✓	✓	✓	✓	✓	✓	✓	✗	✗	✓	✓	✗
Estado	Fallo	Fallo	Fallo	Fallo	Fallo	Fallo	Fallo	Hit	Hit	Fallo	Fallo	Hit

Leyenda:

- ✓ Fallo de página (page fault)
- ✗ No hay fallo (page hit)
- Página reemplazada en ese instante

Resultado:

6 fallos de página (mínimo posible)

¿Cómo decide OPT?

- Cuando ocurre un fallo, reemplaza la página cuya próxima referencia está más lejana en el futuro.
- Si una página no volverá a usarse, es la primera en ser reemplazada.

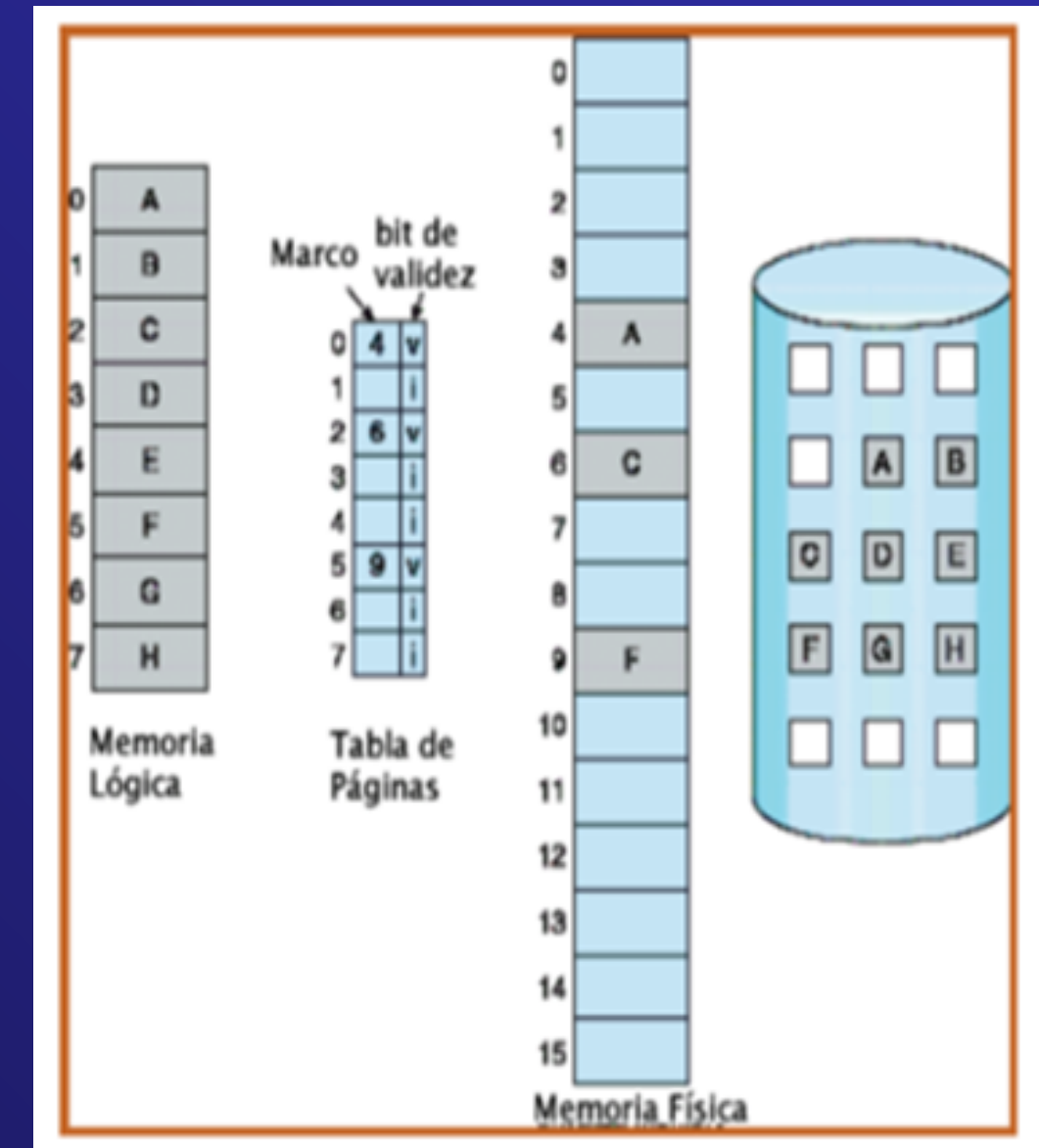
- Propuesto por Bélády (1966). Reemplaza la página que no será utilizada por el mayor tiempo en el futuro.
- Garantiza el mínimo número posible de fallos de página para cualquier cadena de referencias con n marcos dados.
- No es implementable en sistemas reales porque requiere conocimiento del futuro.
- Uso: sirve como cota inferior teórica para evaluar el rendimiento de otros algoritmos. Si LRU está cerca de OPT, es suficientemente bueno.
- Mismo ejemplo → 6 fallos (mínimo posible).



Paginación bajo Demanda y Fallos de Página

¿Qué es la Paginación bajo Demanda?

- Estrategia donde las páginas de un proceso no se cargan en RAM al inicio, sino solo cuando el proceso las referencia por primera vez.
- Combina el concepto de memoria virtual con la gestión lazy (perezosa) de recursos.
- Analogía: es como un libro en una biblioteca — no se fotocopia entero antes de leerlo, solo se saca la página que se necesita.



Beneficios de la Paginación Bajo Demanda

Solo se cargan en memoria las páginas que realmente se necesitan.

CARGA COMPLETA (SIN PAGINACIÓN BAJO DEMANDA)

Espacio de direcciones
del proceso

1
2
3
4
5
6
7
8
9
10
11
12
...
n

RAM

Marco 1	
Marco 2	
Marco 3	
Marco 4	
Marco 5	
Marco 6	
Marco 7	
Marco 8	
	⋮

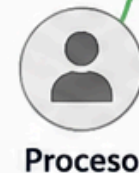


Se cargan todas las páginas
aunque muchas no se usen.

PAGINACIÓN BAJO DEMANDA

Espacio de direcciones
del proceso

1
2
3
4
5
6
7
8
9
10
11
12
...
n



Proceso

Solo se cargan
las páginas
necesarias

RAM

Marco 1	2
Marco 2	5
Marco 3	8
Marco 4	
Marco 5	
Marco 6	
Marco 7	
Marco 8	
	⋮



Se cargan solo las páginas
que el proceso utiliza.

BENEFICIOS PRINCIPALES



Mayor eficiencia en el uso de memoria

La RAM se usa para las páginas realmente necesarias.



Permite ejecutar procesos más grandes que la RAM

Ideal para aplicaciones grandes y sistemas con memoria limitada.



Mejora el rendimiento global

Menos tiempo cargando datos innecesarios, más tiempo en ejecución útil.



Soporte para multiprogramación

Más procesos pueden residir en memoria al mismo tiempo.



Aislamiento y protección

Cada página se maneja de forma independiente, mejorando la seguridad y estabilidad del sistema.



Idea clave:

La paginación bajo demanda trae a memoria una página solo cuando se necesita (fallo de página) y la reemplaza cuando es necesario (algoritmo de reemplazo).

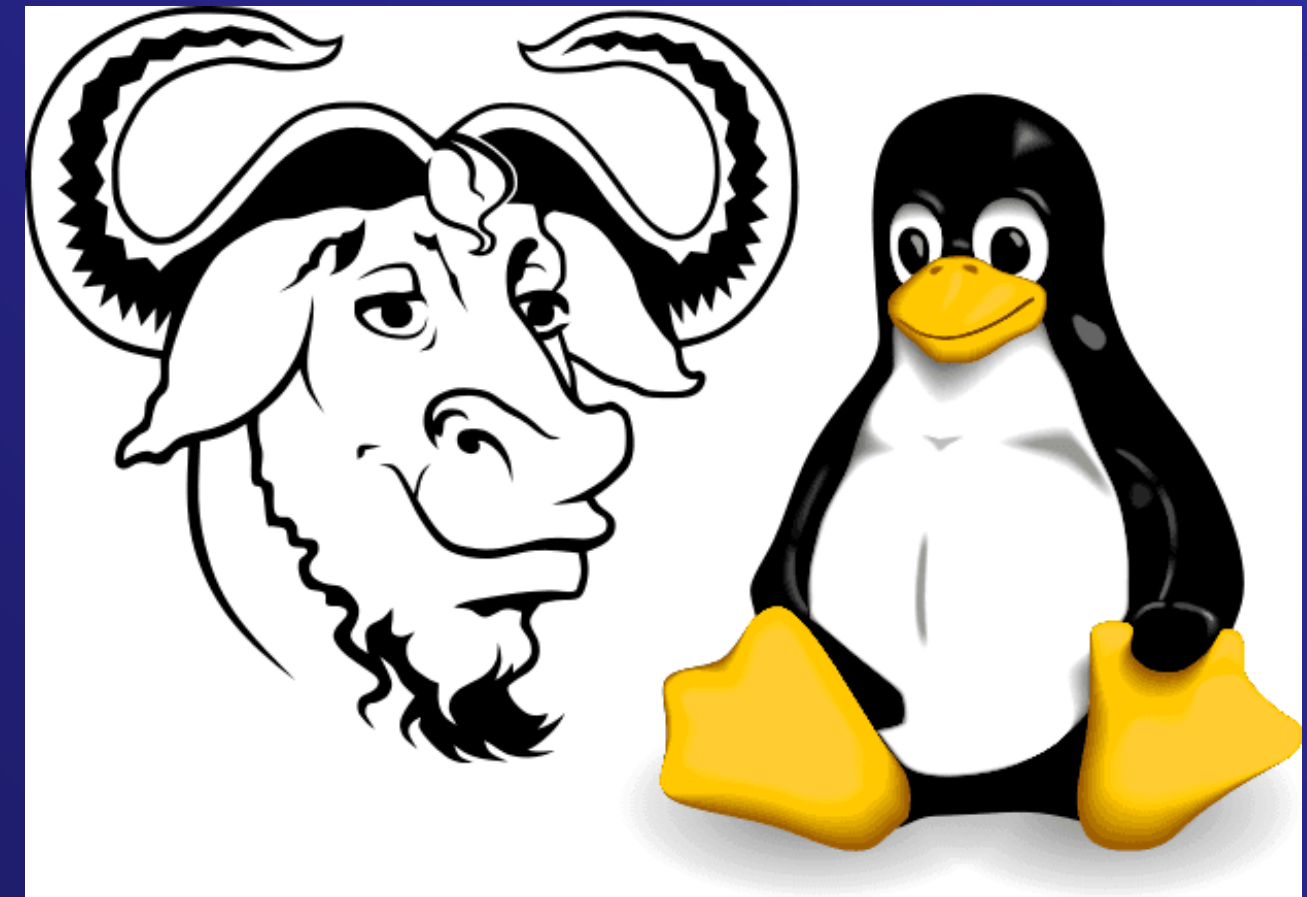
Linux vs GNU/Linux

A menudo llamado simplemente "Linux", pero es una combinación:

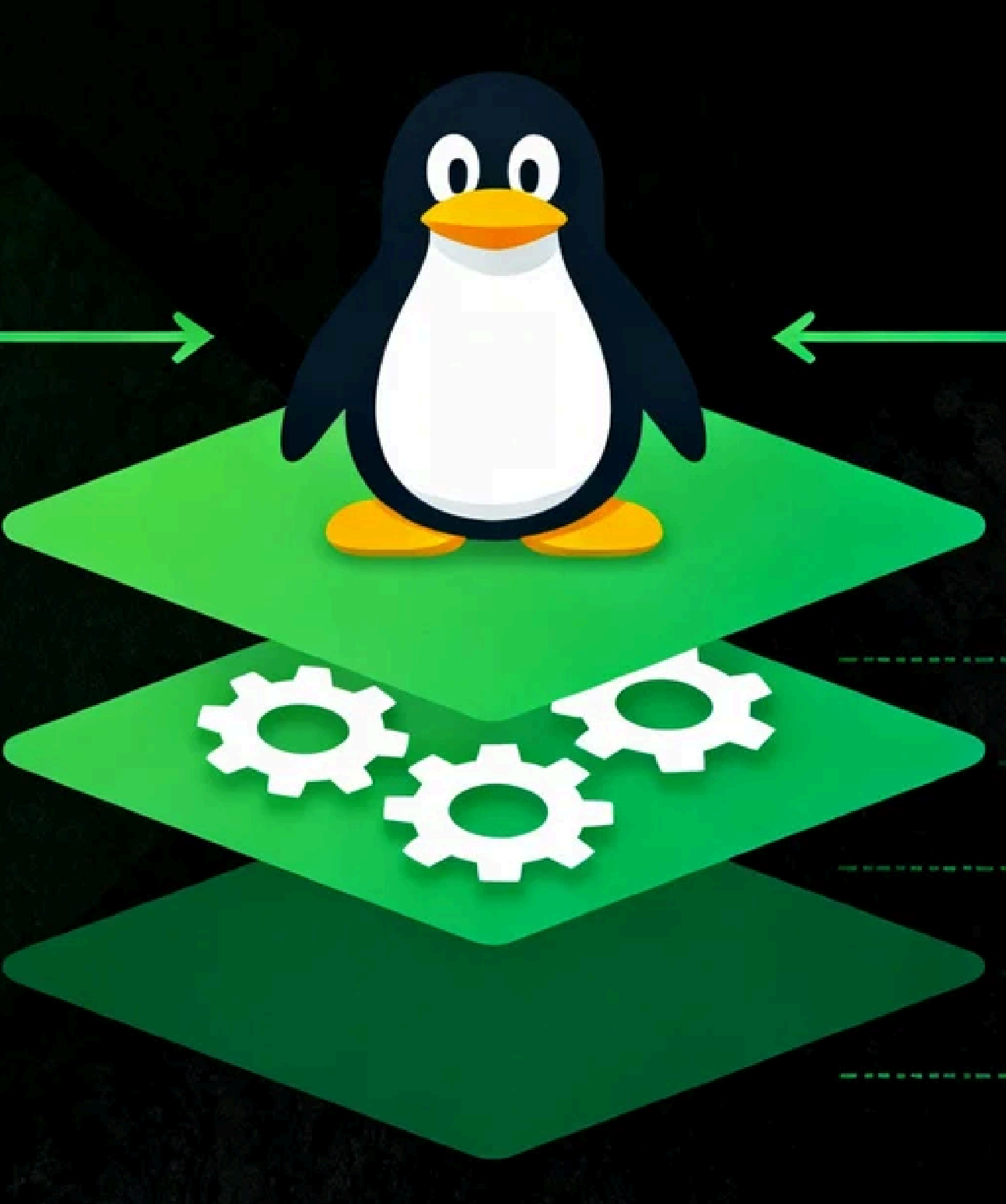
Linux: Es el nombre del Kernel (Núcleo).

GNU: Proyecto que aporta herramientas (compiladores, shells, editores).

* Filosofía: Software libre y de código abierto (FOSS).



Cabe señalar que existen derivados que usan el núcleo Linux pero que no tienen componentes GNU, como el sistema operativo Android, así como también existen distribuciones de software GNU donde el núcleo Linux está ausente como FreeBSD.



Arquitectura del Kernel

Kernel de Linux.

Estructura del código fuente del Kernel.

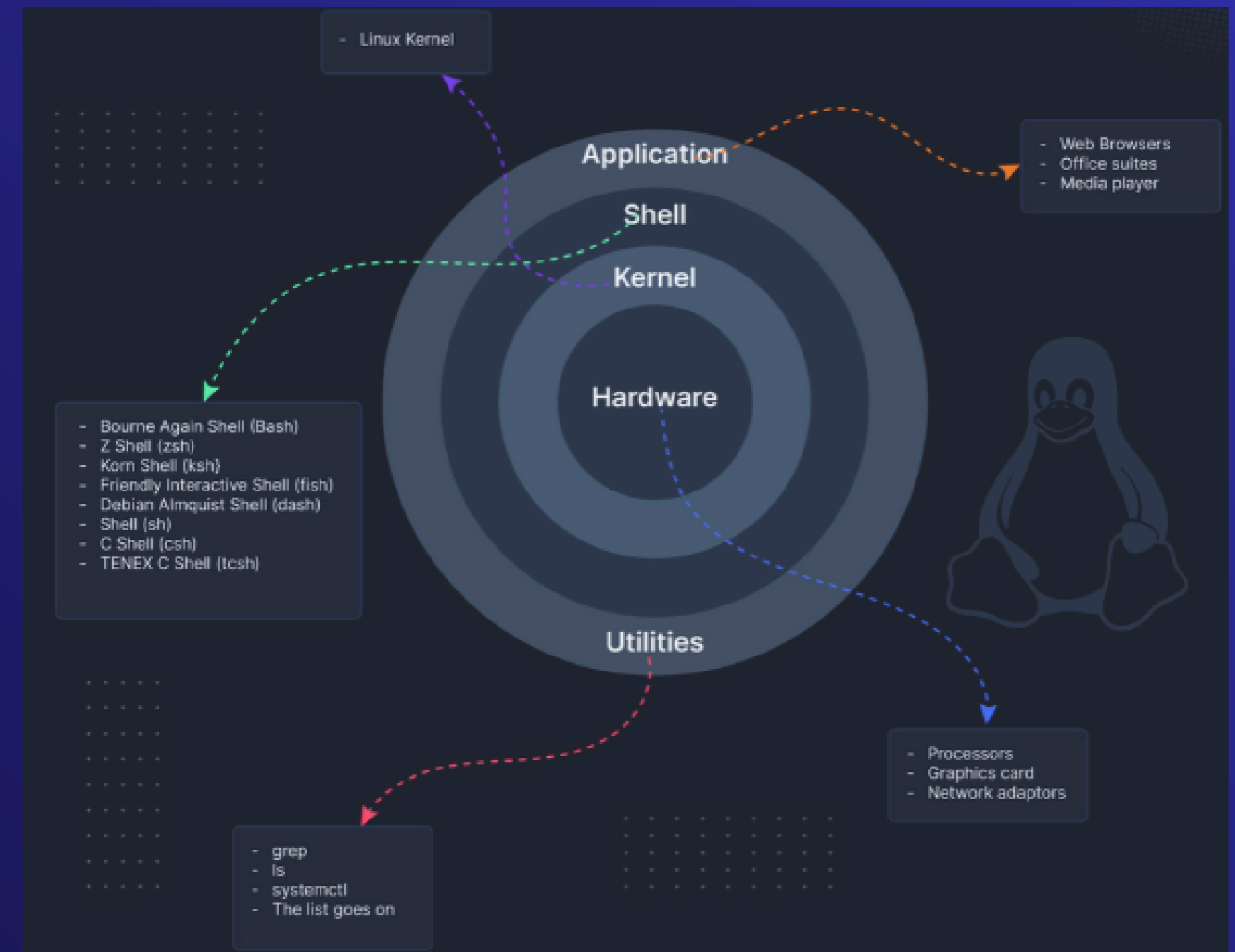
Escribir y compilar módulos del kernel (LKM).

Carga y descarga de módulos dinámicos.

Kernel de Linux

Actúa como la interfaz principal entre el hardware y los procesos.

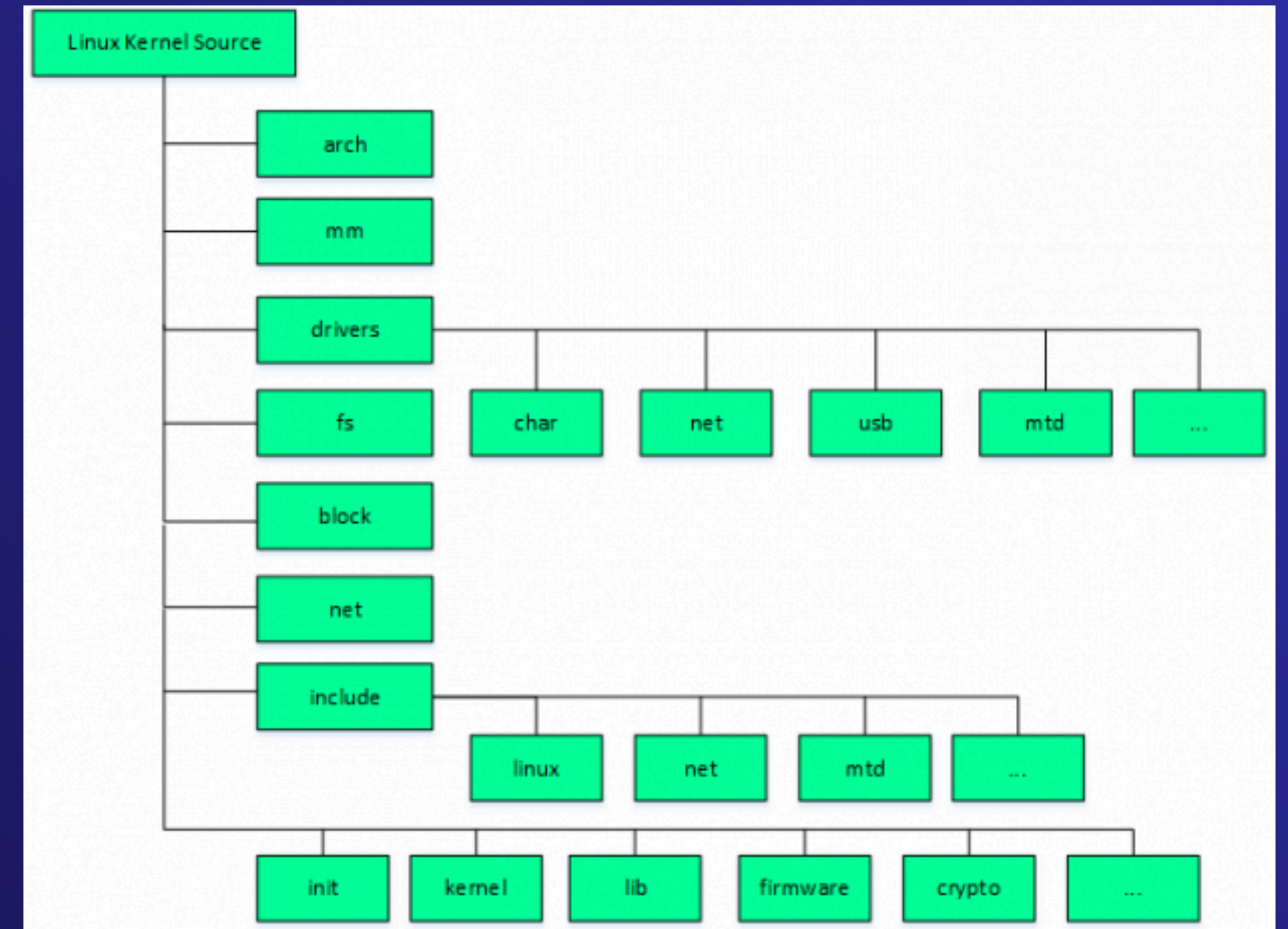
- Gestor de Procesos: Planificación y multitarea.
- Gestión de Memoria: Paginación y segmentación.
- Subsistema de Archivos: Interfaz común (VFS).
- Syscalls: API para solicitudes del usuario.



Estructura del código fuente del Kernel

El código fuente del kernel de Linux está organizado en varios subdirectorios dentro del árbol de código principal, cada uno dedicado a un subsistema específico.

Esta estructura modular permite mantener el código bien organizado y facilita el mantenimiento y desarrollo de nuevas características.



/arch: Contiene el código específico para diferentes arquitecturas de hardware (como x86, ARM, MIPS, etc.). Aquí se encuentran las configuraciones y adaptaciones del kernel para funcionar en diferentes plataformas.

/kernel: Incluye el núcleo del código del sistema operativo, con funcionalidades como la gestión de procesos, sincronización, y planificación. Es el corazón del sistema y maneja la lógica central del kernel.

/include: Contiene los archivos de cabecera utilizados por el resto del kernel. Estos archivos definen estructuras de datos, funciones, y constantes usadas a lo largo del código.

/fs: Código relacionado con los sistemas de archivos soportados por Linux. Aquí se implementan los diferentes tipos de sistemas de archivos y el VFS (Virtual File System).

/drivers: Contiene los controladores de dispositivos para el hardware, como controladores de red, de almacenamiento, de video, y otros periféricos. Esta estructura modular permite que los drivers se carguen y descarguen dinámicamente, mejorando la adaptabilidad del kernel.

/net: Incluye el código para la pila de redes y los protocolos de comunicación, como TCP/IP, IPv4, IPv6, y otros. Este directorio es fundamental para todas las funcionalidades de red del kernel.

/mm: Contiene el código de gestión de memoria, incluyendo algoritmos de administración de memoria virtual, paginación, intercambio (swapping) y asignación de memoria. Esta sección es crucial para optimizar el uso de recursos y evitar cuellos de botella.

/security: Código relacionado con la seguridad del sistema, como el módulo SELinux, AppArmor, y otros mecanismos de control de acceso. También se ocupa de la implementación de políticas de seguridad para proteger el sistema de amenazas.

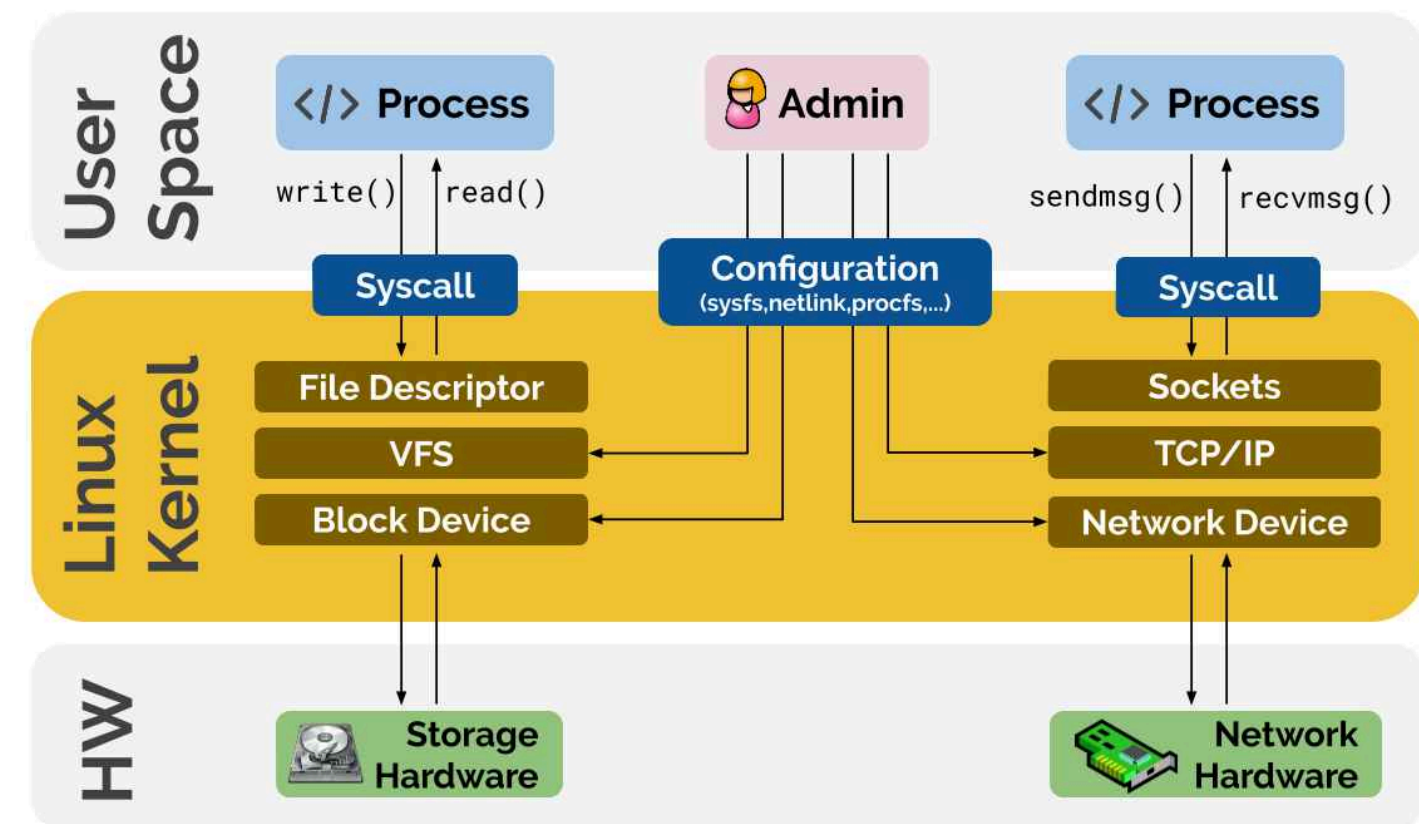
/init: Contiene el código de inicialización del kernel que se ejecuta al inicio del sistema. Este código prepara el entorno y carga las configuraciones necesarias para que el kernel pueda iniciar correctamente.

Módulos del kernel (LKM)

Los módulos del kernel son componentes de código que se pueden cargar y descargar en tiempo de ejecución, lo que permite extender las funcionalidades del kernel sin necesidad de recompilar el sistema.

Esto proporciona flexibilidad y facilidad de mantenimiento al sistema operativo.

Kernel Architecture



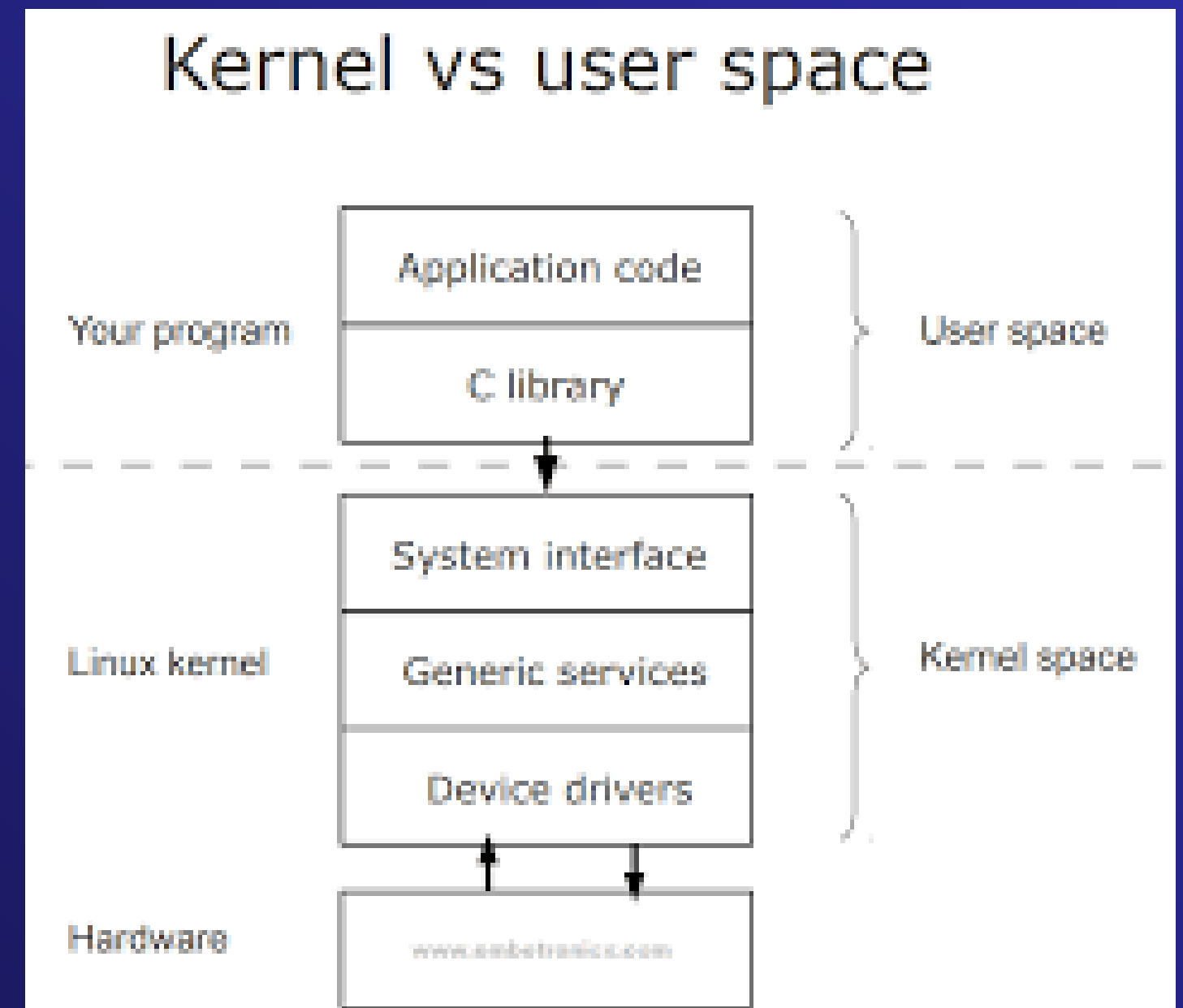
Espacios del Kernel

Kernel

Entorno privilegiado con acceso completo al hardware. Aquí reside el núcleo y los módulos críticos.

Usuario

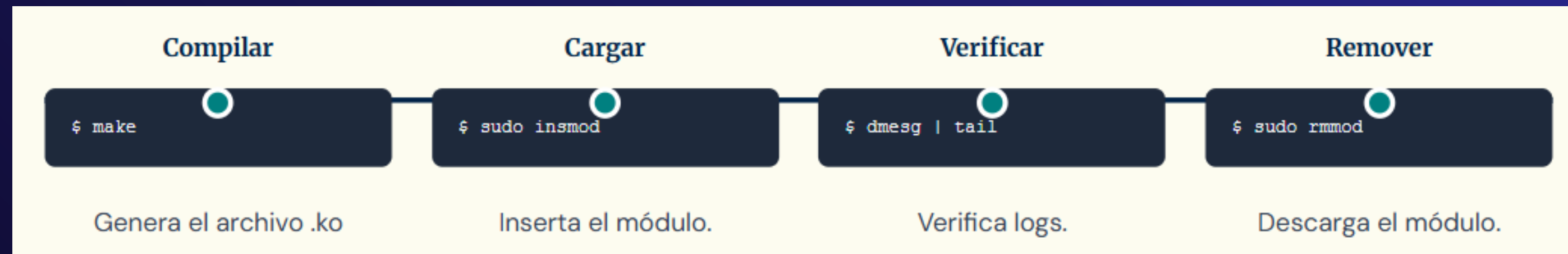
Entorno restringido donde corren las aplicaciones. Acceso limitado al hardware para garantizar estabilidad.



Módulos Cargables LKM

Los Loadable Kernel Modules permiten extender las funcionalidades del kernel sin necesidad de recompilar todo el sistema operativo.

Ideales para controladores de hardware y sistemas de archivos.



Escribir y compilar módulos del kernel

Archivo modulo.c

```
1  #include <linux/module.h> // Requerido para todos los módulos
2  #include <linux/kernel.h> // Requerido para KERN_INFO
3  #include <linux/init.h>   // Requerido para las macros init y exit
4
5
6  static int __init hola_kernel_init(void) {
7      printk(KERN_INFO "Módulo cargado exitosamente. Hola desde el Kernel.\n");
8      return 0;
9  }
10
11
12 static void __exit hola_kernel_exit(void) {
13     printk(KERN_INFO "Módulo descargado. Adiós mundo kernel.\n");
14 }
15
16
17 module_init(hola_kernel_init);
18 module_exit(hola_kernel_exit);
19
20 MODULE_LICENSE("GPL");
21 MODULE_AUTHOR("Sistemas Operativos 2");
22 MODULE_DESCRIPTION("Un módulo simple para aprender LKM");
```

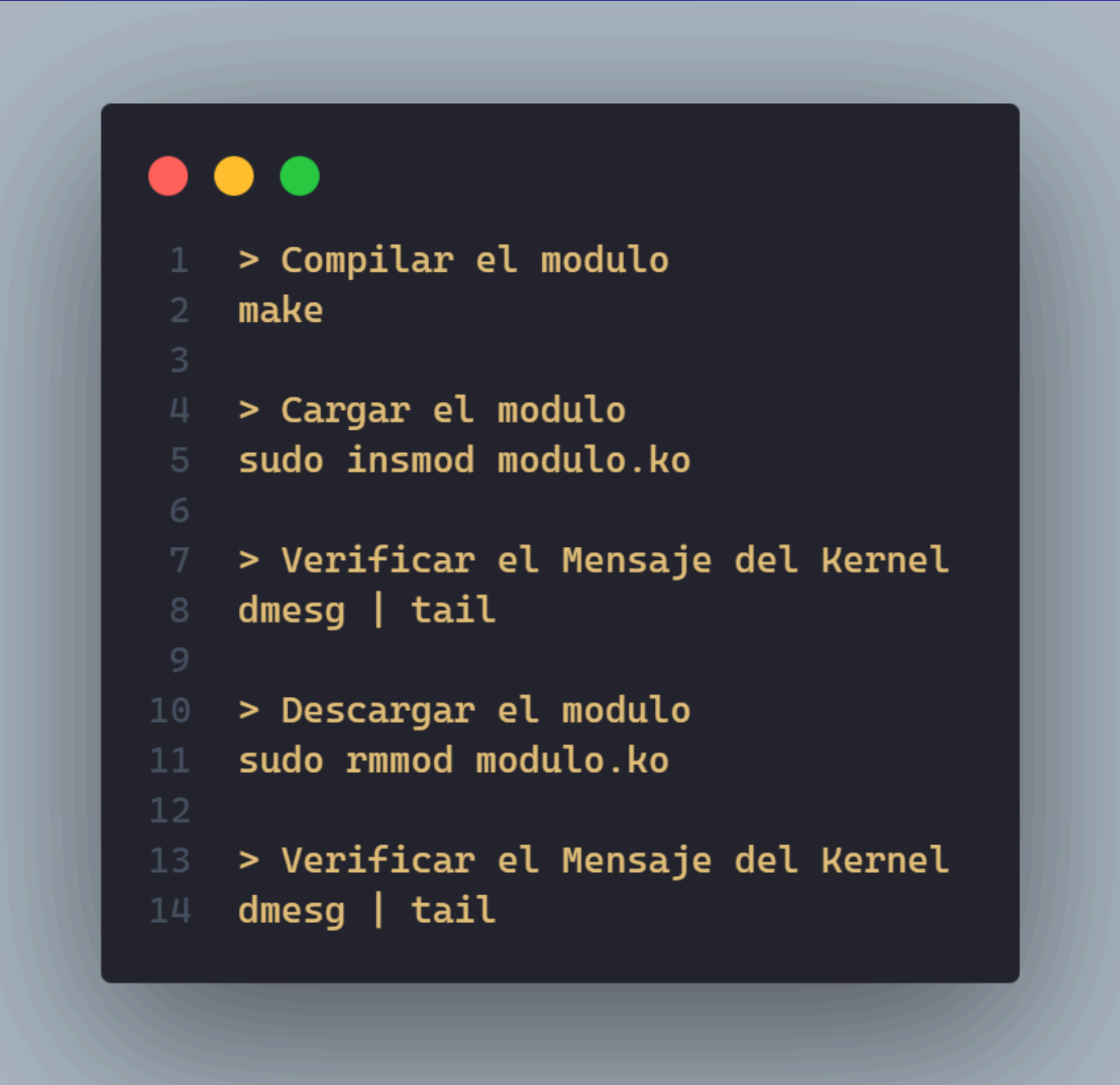
Archivo makefile

```
1  obj-m += modulo_hola.o
2
3  all:
4      make -C /lib/modules/$(shell uname -r)/build M=$(PWD) modules
5
6  clean:
7      make -C /lib/modules/$(shell uname -r)/build M=$(PWD) clean
```

Para compilar el nuevo modulo se necesita abrir una terminal y navegar al directorio donde se alojan los archivos `hello.c` y `Makefile`. Luego, ejecutar los siguientes comando para compilar el módulo

Nota: No olvidar instalar dependencias si aun no están instaladas:

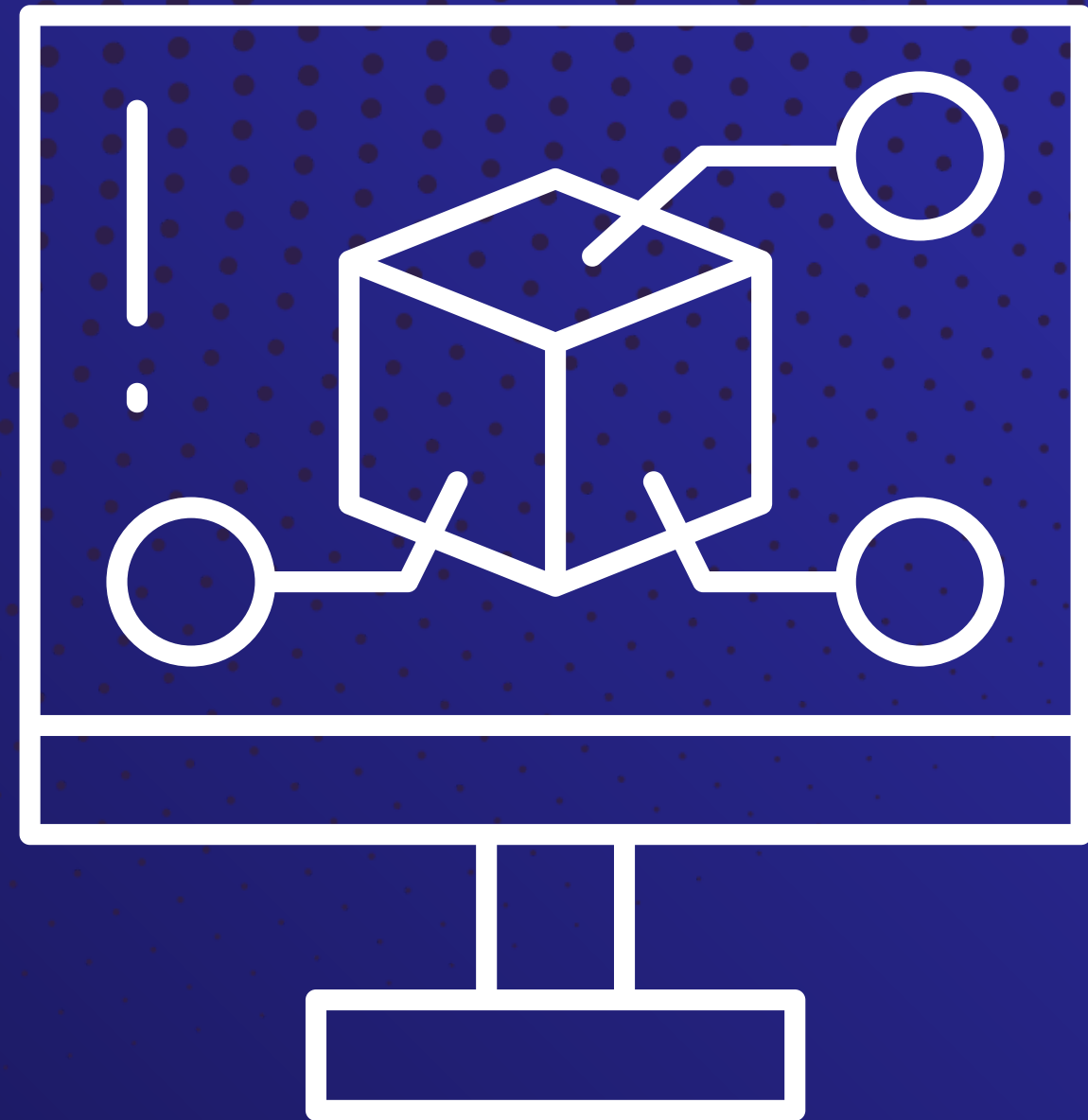
```
sudo apt -y install build-essential libncurses-dev bison  
flex libssl-dev libelf-dev
```



```
1  > Compilar el modulo  
2  make  
3  
4  > Cargar el modulo  
5  sudo insmod modulo.ko  
6  
7  > Verificar el Mensaje del Kernel  
8  dmesg | tail  
9  
10 > Descargar el modulo  
11 sudo rmmod modulo.ko  
12  
13 > Verificar el Mensaje del Kernel  
14 dmesg | tail
```

**Ejemplo
Guiado:**

**Creación módulo
LKM**





¿DUDAS?

Gracias por la atención

**RECUERDA QUE TENEMOS NUESTRO FORO SEMANAL DONDE
PUEDES CONSULTAR CUALQUIER DUDA QUE TE SURJA EN LA
SEMANA**